MODULE #2

Dr Suriya Prakash J

Binary Search

- Binary Search is a searching algorithm for finding an elements position in a sorted array.
- It uses Divide and Conquer method to identify the solution.
- Divide and Conquer method mean by splitting the entire array in to two half and further splitting the two half until finding solution.
- The Key value will be compared with the mid element of the Array . If the match is found then it will return the array position as output.

Binary Search

0	1	2	3	4	5	6	7	8
10	**12**	**24**	**29**	**39**	**40**	**51**	**56**	**69**
↑								↑
Beg			K=56					End

mid=(beg+end)/2
beg=0
end=8
mid=(0+8)/2 =>4

Binary Search

0	1	2	3	4	5	6	7	8
10	**12**	**24**	**29**	**39**	**40**	**51**	**56**	**69**
↑ beg		K=56		↑ mid				↑ end

mid=4
A[mid]=39

A[mid]<K
beg=mid+1 ,end=8
A[mid]>k
end=mid-1 ,beg=0
A[mid]==k
Return Array position

Binary Search

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69
					↑	↑		↑
		K=56			beg	mid		end

beg=5
end=8
mid=(5+8)/2 =>6

mid=6
A[mid]=51

A[mid]<K
beg=mid+1 ,end=8

Binary Search

0	1	2	3	4	5	6	7	8
10	12	24	29	39	40	51	56	69
							↑ beg ↑ mid	↑ end

K=56

beg=7
end=8
mid=(7+8)/2 =>7

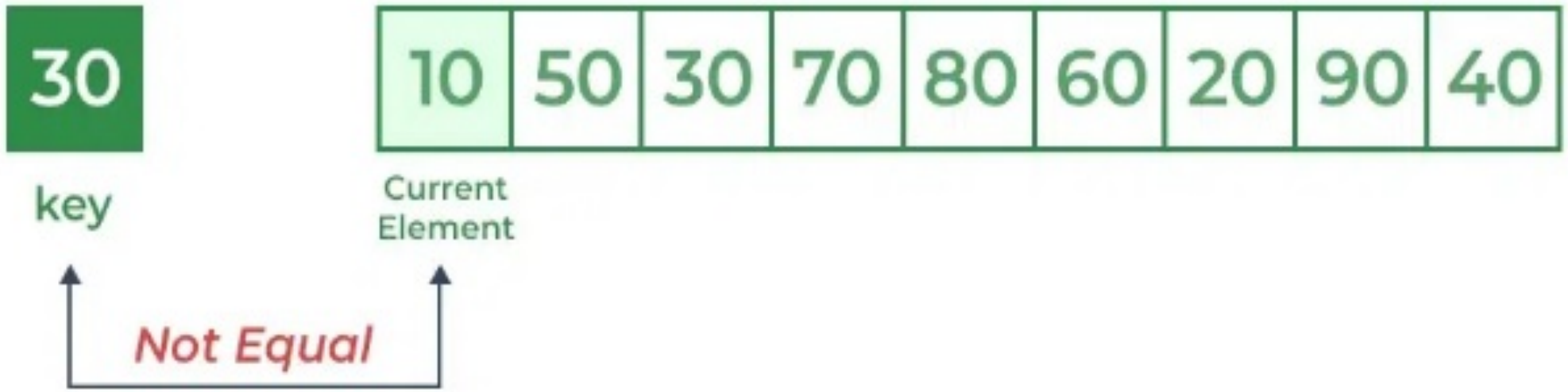
mid=7
A[mid]=56

A[mid]==K
Return the position 7

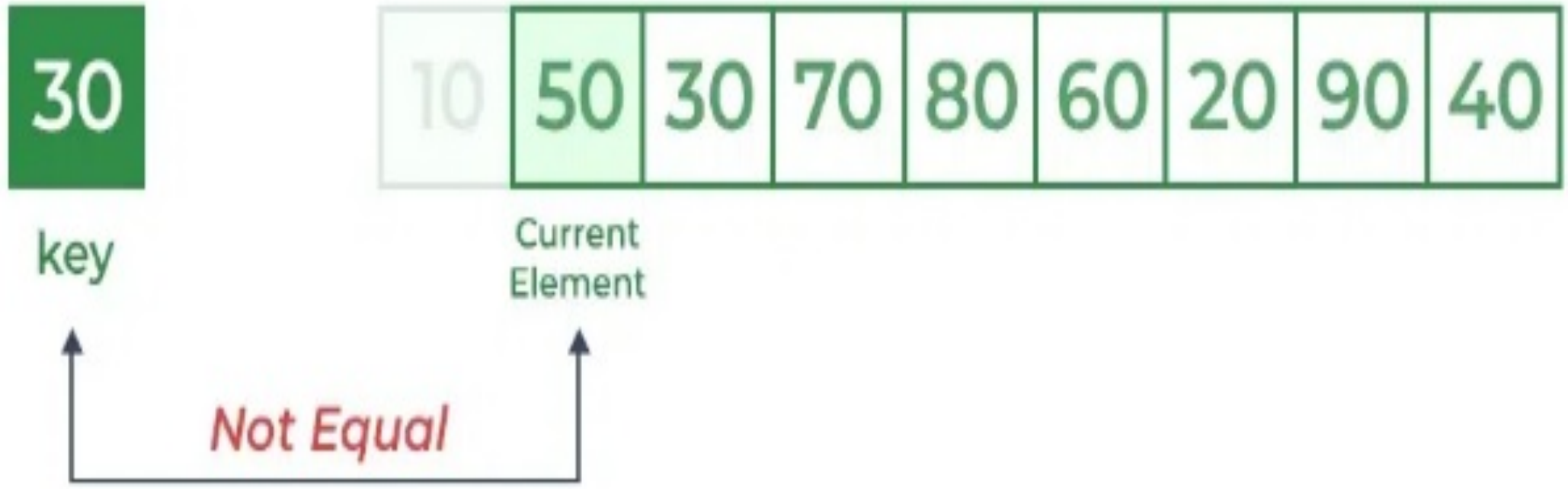
Linear Search

- ***Linear Search*** *is defined as a sequential search algorithm that starts at one end and goes through each element of a list until the desired element is found, otherwise the search continues till the end of the data set.*
- ***Step 1:*** *Start from the first element (index 0) and compare* ***key*** *with each element (arr[i]).*

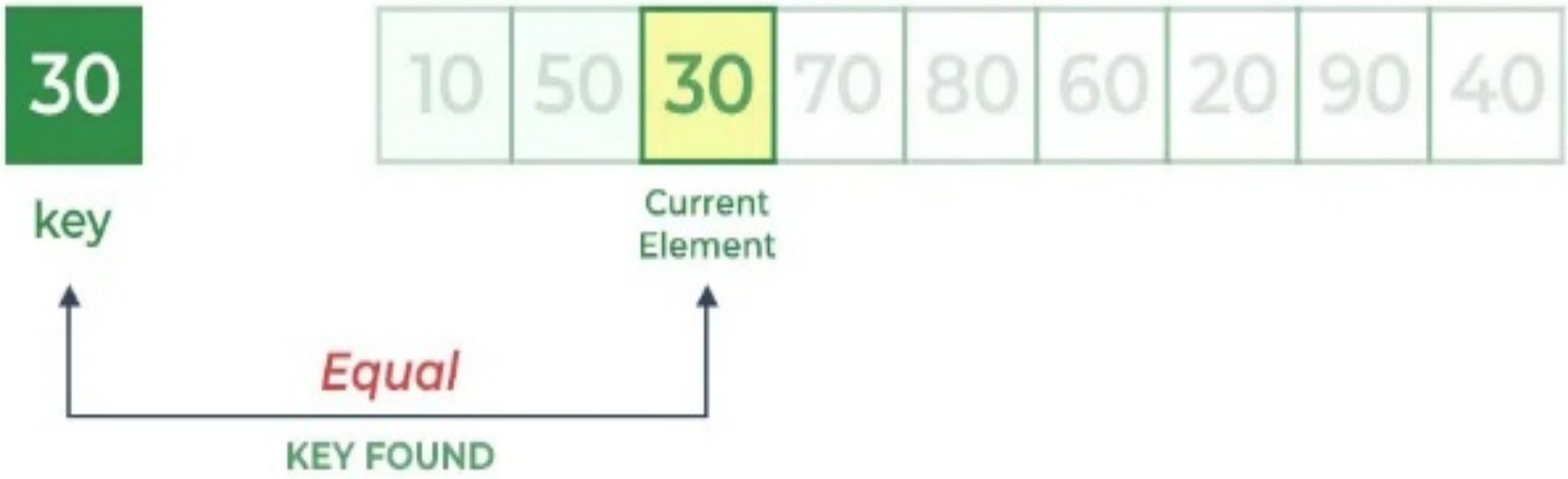
Comparing key with first element arr[0]. Since not equal, the iterator moves to the next element as a potential match.



Comparing key with next element arr[1]. Since not equal, the iterator moves to the next element as a potential match.



Step 2: *Now when comparing arr[2] with key, the value matches. So the Linear Search Algorithm will yield a successful message and return the index of the element when key is found (here 2).*



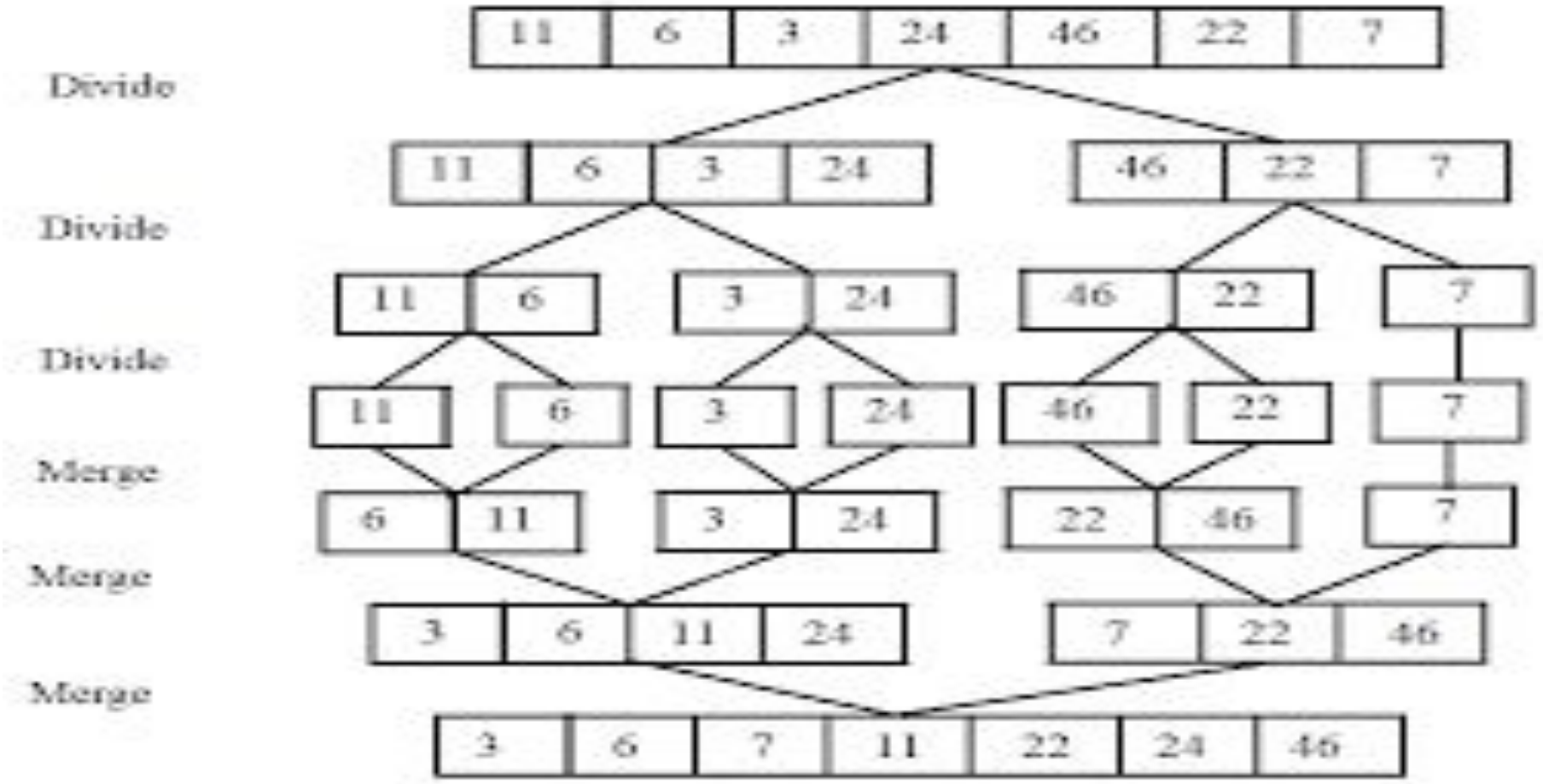
MergeSort Algorithm

Merge sort divide and conquer approach to sort the elements. It is one of the most popular and efficient sorting algorithm. It divides the given list into two equal halves, calls itself for the two halves and then merges the two sorted halves. We have to define the **merge()** function to perform the merging.

The sub-lists are divided again and again into halves until the list cannot be divided further. Then we combine the pair of one element lists into two-element lists, sorting them in the process. The sorted two-element pairs is merged into the four-element lists, and so on until we get the sorted list.

mid = (beg + end)/2

Merge Sort


Divide
Divide
Divide
Merge
Merge
Merge
11 6 3 24 46 22 7
11 6 3 24
46 22 7
11 6
3 24
46 22
7
11
6
3
24
46
22
7
6 11
3 24
22 46
7
3 6 11 24
7 22 46
3 6 7 11 22 24 46

Quick sort Algorithm

- Quicksort is the widely used sorting algorithm that makes **n log n** comparisons in average case for sorting an array of n elements. It is a faster and highly efficient sorting algorithm. This algorithm follows the divide and conquer approach. Divide and conquer is a technique of breaking down the algorithms into subproblems, then solving the subproblems, and combining the results back together to solve the original problem.
- Quicksort picks an element as pivot, and then it partitions the given array around the picked pivot element. In quick sort, a large array is divided into two arrays in which one holds values that are smaller than the specified value (Pivot), and another array holds the values that are greater than the pivot.
- After that, left and right sub-arrays are also partitioned using the same approach. It will continue until the single element remains in the sub-array.
- <u>Selection of Pivot Element:</u>
- Pivot can be random, i.e. select the random pivot from the given array.
- Pivot can either be the rightmost element of the leftmost element of the given array.
- Select median as the pivot element.

Quick Sort Algorithm

1) If A[Pivot] < A[Right]

   Right--;

   (Move the Right Position Towards Left Side)

2) If A[Pivot] > A[Right]

   Swap Pivot Element with the Right Element and move Pivot Position to Right position.

Quick Sort Algorithm

1) If A[Pivot] > A[Left]

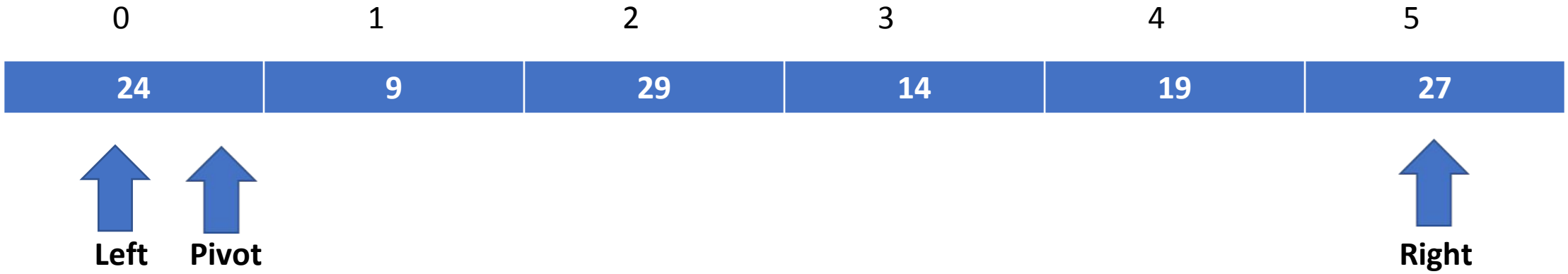
   Left++;

   (Move the Left Position Towards Right Side)

2) If A[Pivot] < A[Left]

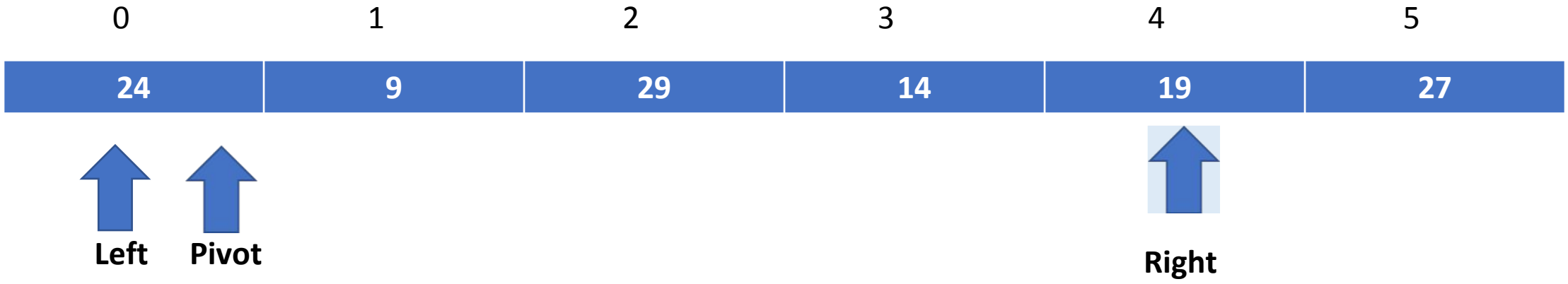
   Swap Pivot Element with the Left Element and move Pivot Position to Left position.

Quick Sort Algorithm



A[Pivot]=24
A[Left]=24
A[Right]=27

Quick Sort Algorithm



If A[Pivot] < A[Right]

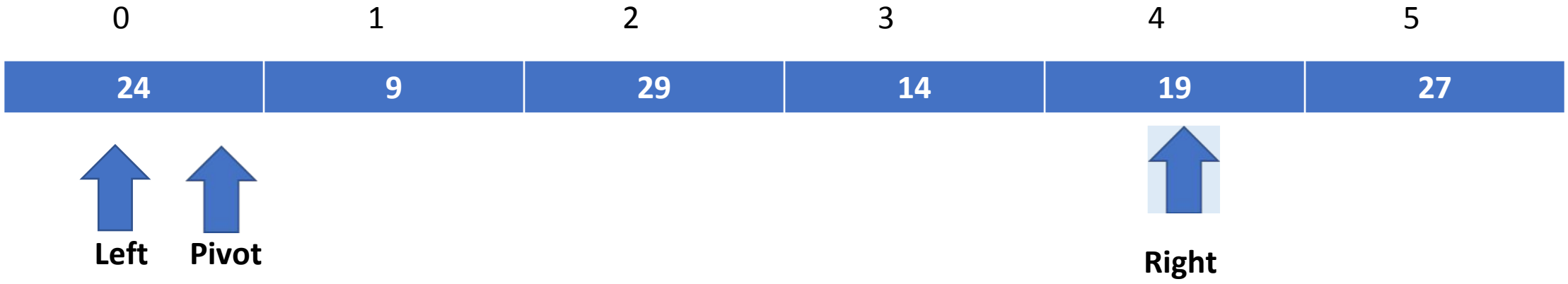
Right--;

(Move the Right Position Towards Left Side)

If 24<27

Right--;

Quick Sort Algorithm



If A[Pivot] < A[Right]

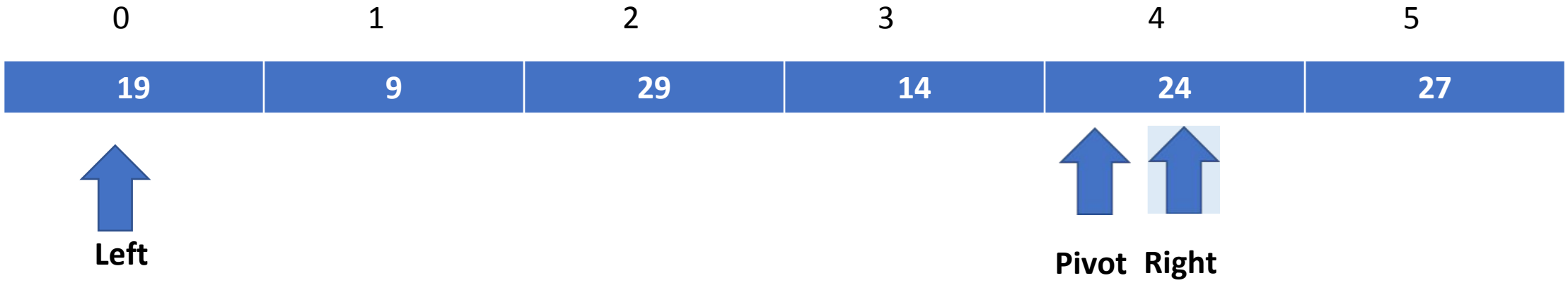
Right--;

(Move the Right Position Towards Left Side)

If 24<19 (condition false)

Right--;

Quick Sort Algorithm



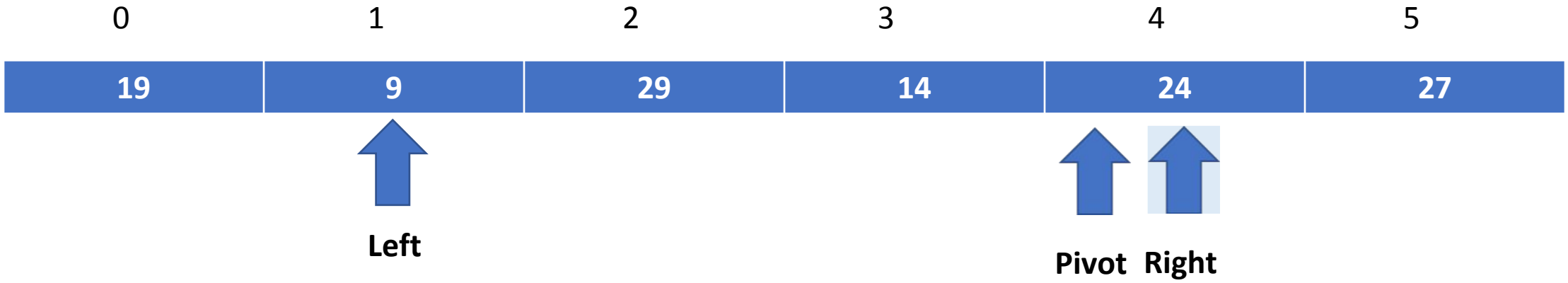
If A[Pivot] > A[Right]

(Swap Pivot Element with the Right Element and move Pivot Position of Right position.)

If 24>19

Swap Pivot Element 24 with the Right Element 19 and move Pivot Position to Right position.)

Quick Sort Algorithm



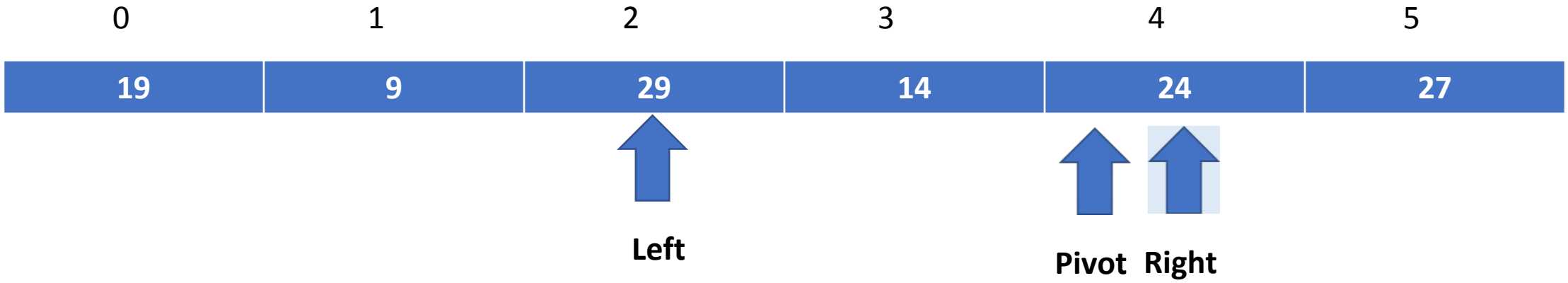
If A[Pivot] > A[Left]

    Left++;

If 24>19

    Left++;

Quick Sort Algorithm



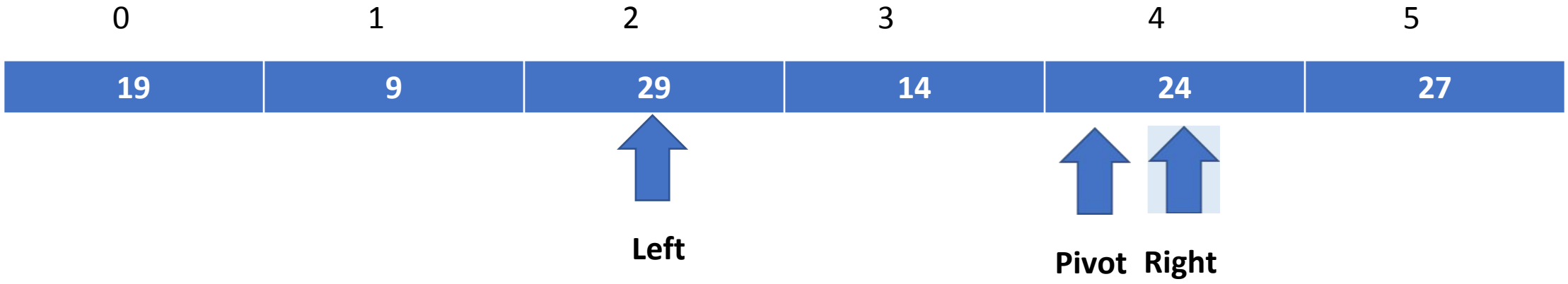
If A[Pivot] > A[Left]

    Left++;

If 24>9

    Left++;

Quick Sort Algorithm



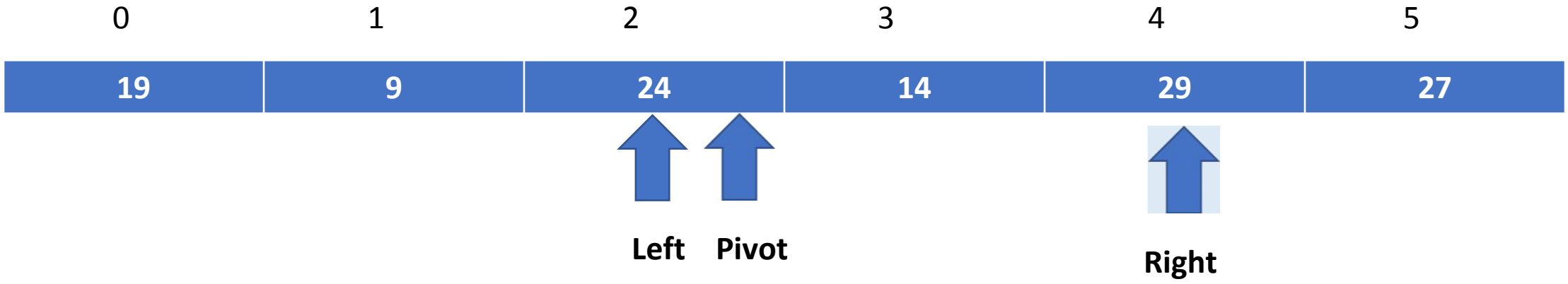
If A[Pivot] > A[Left]

Left++;

If 24>29 (condition False)

Left++;

Quick Sort Algorithm



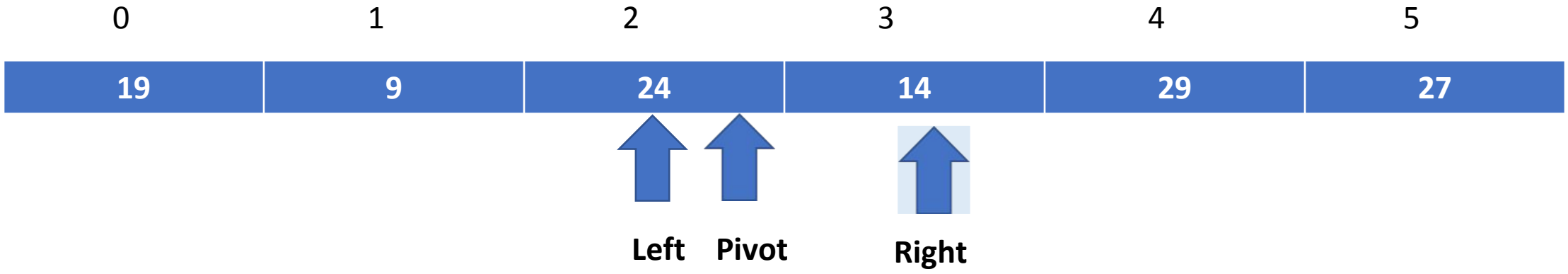
If A[Pivot] < A[Left]

Swap Pivot Element with the Left Element and move Pivot Position to Left position.

If 24<29

Swap Pivot Element 24 with the Left Element 29 and move Pivot Position to Left position.

Quick Sort Algorithm



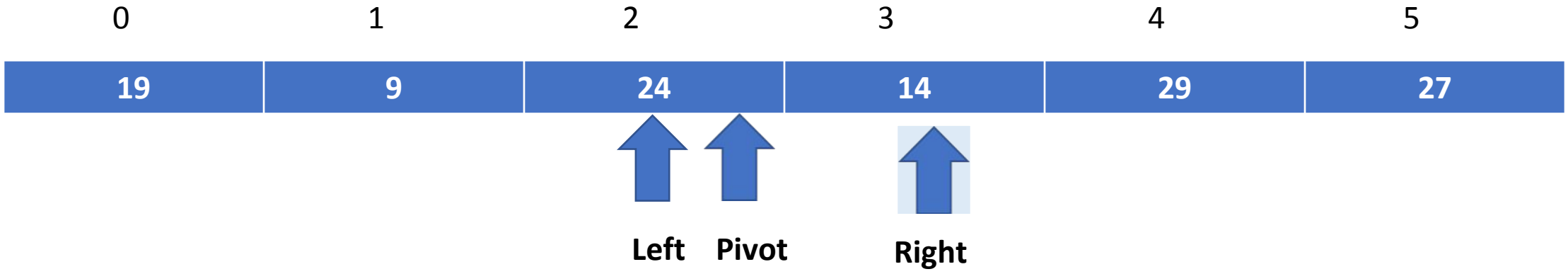
If A[Pivot] < A[Right]

Right--;

If 24<29

Right--;

Quick Sort Algorithm



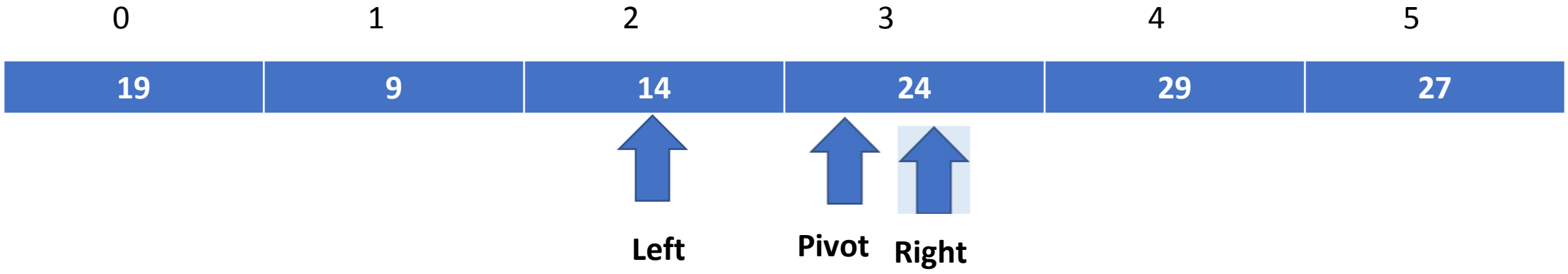
If A[Pivot] < A[Right]

Right--;

If 24<14 (Condition False)

Right--;

Quick Sort Algorithm



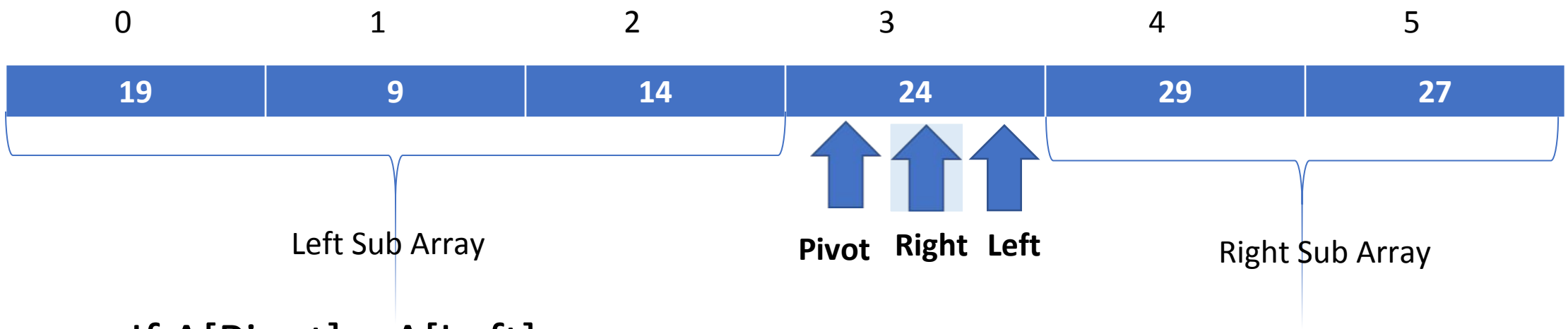
If A[Pivot] > A[Right]

Swap Pivot Element with the Right Element and move Pivot Position to Right position.

If 24>14

Swap Pivot Element 24 with the Right Element 14 and move Pivot Position to Right position.

Quick Sort Algorithm



If A[Pivot] > A[Left]

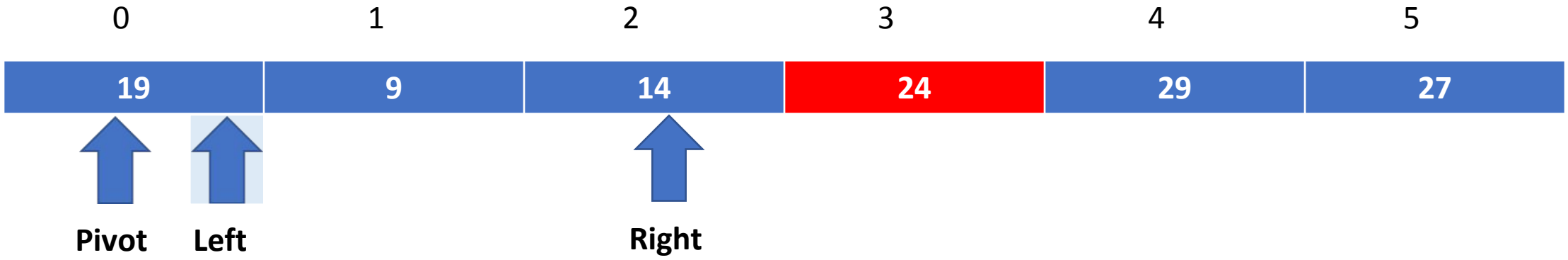
Left++

If 24>14

Left++

In this step All three (left,Right and Pivot) in the same position So we will fix value 24 as sorted value and split **Left sub array** and **right sub array** to continue the same quick sort process.

Quick Sort Algorithm



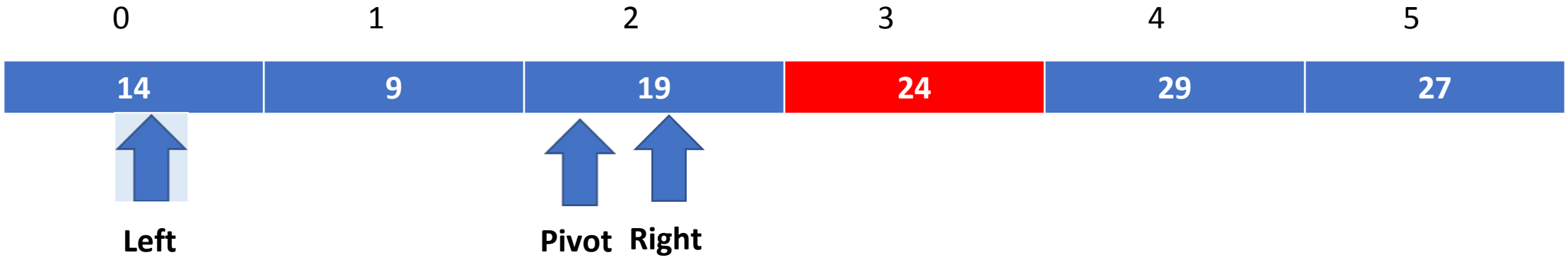
If A[Pivot]<A[Right]

Right--;

If 19<14 (condition False)

Right--;

Quick Sort Algorithm



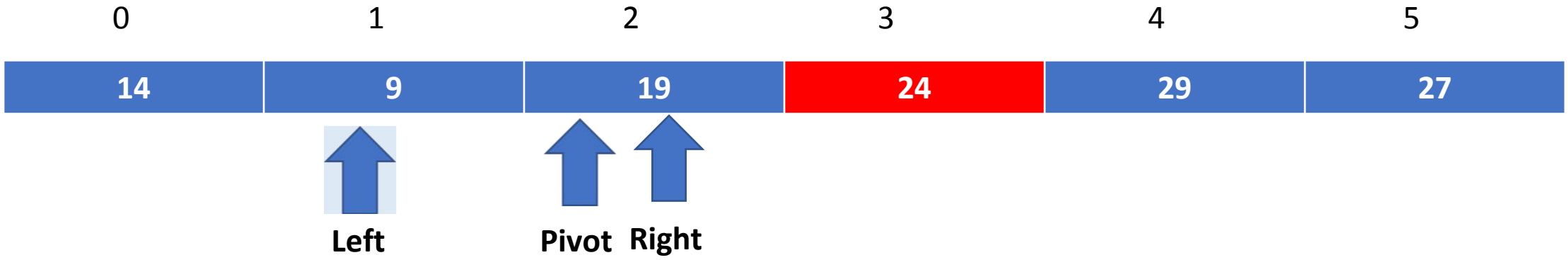
If A[Pivot]>A[Right]

Swap Pivot Element with the Right Element and move Pivot Position to Right position.

If 19>14

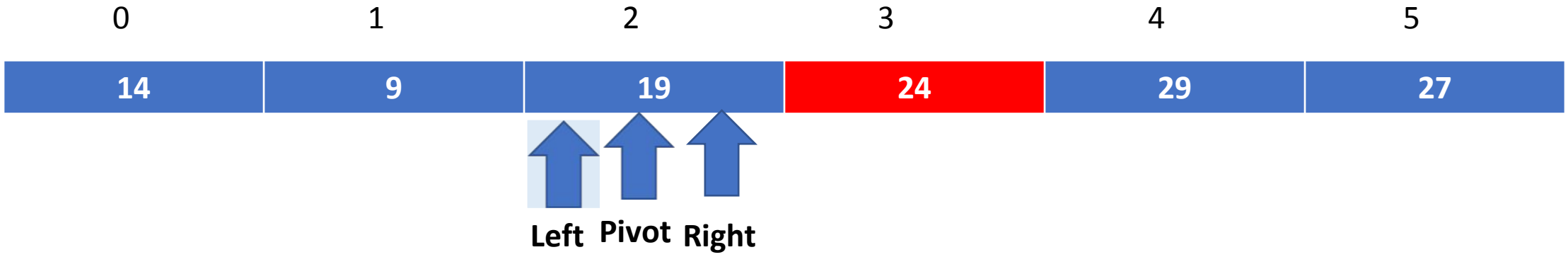
Swap Pivot Element 19 with the Right Element 14 and move Pivot Position to Right position.

Quick Sort Algorithm




```
If A[Pivot]>A[Left]
    Left++;
 If 19>14
 Left++;
```

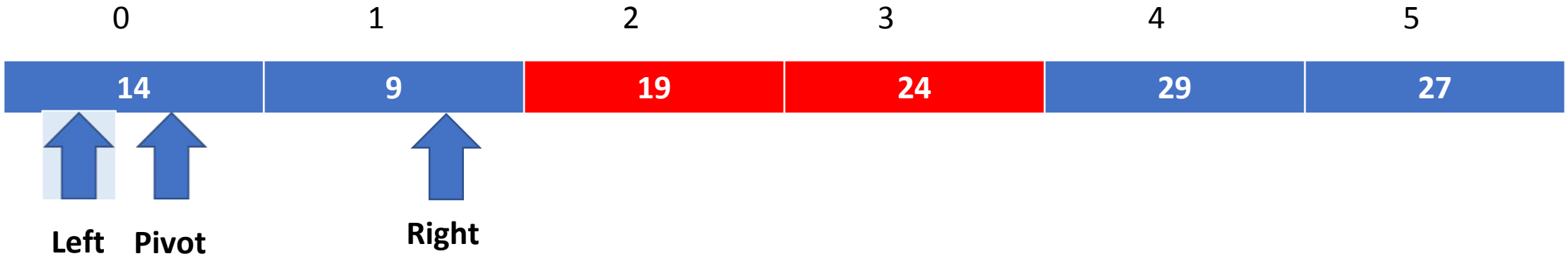
Quick Sort Algorithm




```
If A[Pivot]>A[Left]
    Left++;
 If 19>9
 Left++;
```

In this step All three (left,Right and Pivot) in the same position
So we will fix value 19 as sorted value and split **Left sub array** and **right sub array** to continue the same quick sort process.

Quick Sort Algorithm



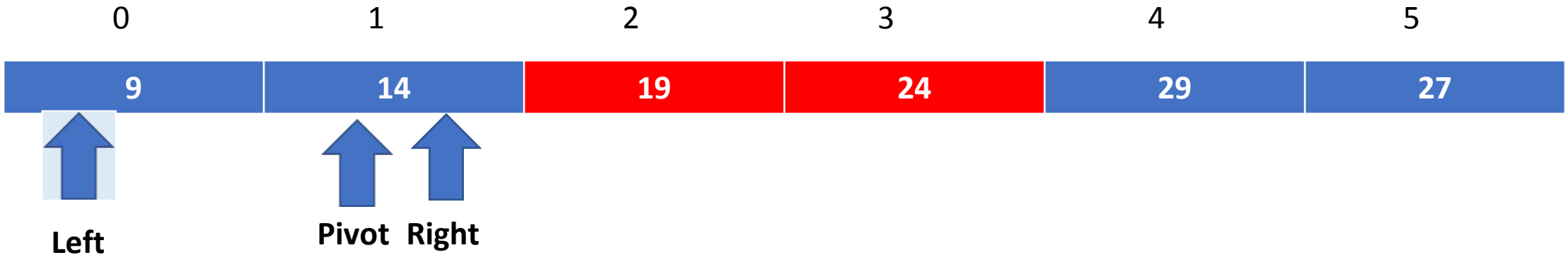
If A[Pivot]<A[Right]

Right--;

If 14<9 (Condition False)

Right--;

Quick Sort Algorithm



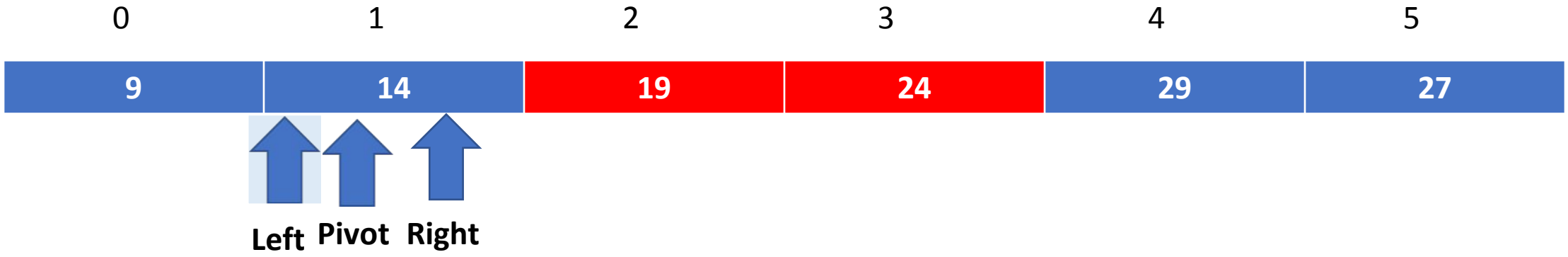
If A[Pivot]>A[Right]

Swap Pivot Element with the Right Element and move Pivot Position to Right position.

If 14<9 (Condition False)

Swap Pivot Element 14 with the Right Element 9 and move Pivot Position to Right position.

Quick Sort Algorithm



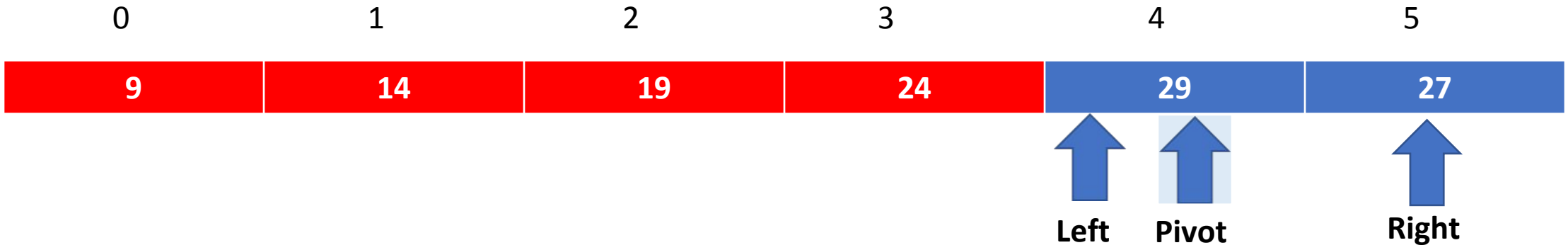
If A[Pivot]>A[Left]

Left++;

If 14>9

Left++;

Quick Sort Algorithm



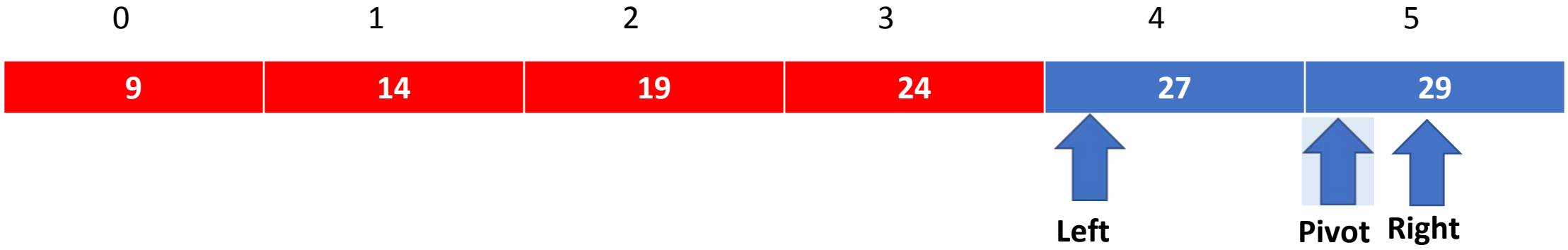
If A[Pivot]>A[Right]

Swap Pivot Element with the Right Element and move Pivot Position to Right position.

If 29>27

Swap Pivot Element 29 with the Right Element 27and move Pivot Position to Right position.

Quick Sort Algorithm



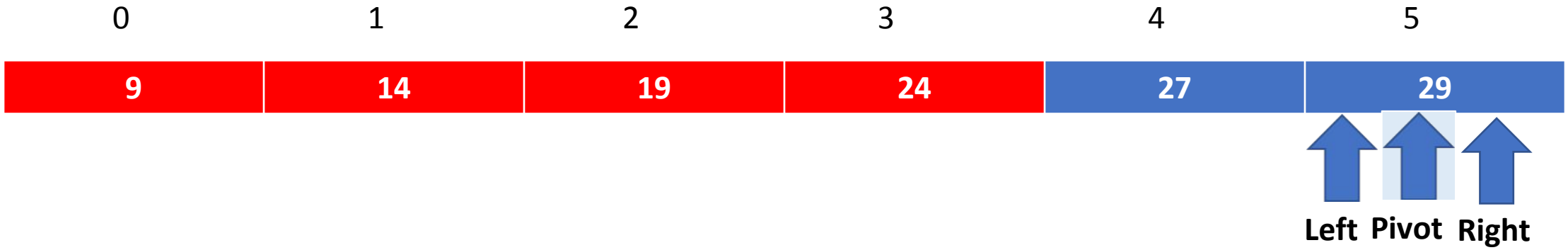
If A[Pivot]<A[Left]

Left++;

If 29>27

Left++;

Quick Sort Algorithm



If A[Pivot]<A[Left]

Left++;

If 29>27

Left++;

Quick Sort Algorithm

0	1	2	3	4	5
9	**14**	**19**	**24**	**27**	**29**

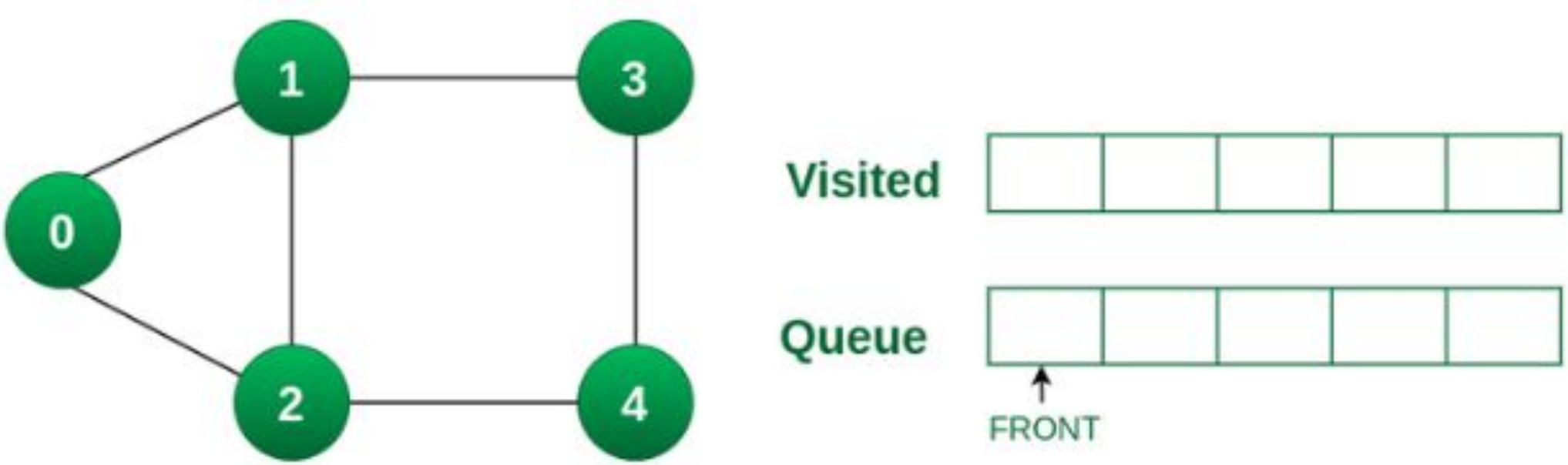
Final sorted array using Quick Sort

BFS (Breadth First Search)

- *The **Breadth First Search (BFS)** algorithm is used to search a graph data structure for a node that meets a set of criteria. It starts at the root of the graph and visits all nodes at the current depth level before moving on to the nodes at the next depth level.*
- **Relation between BFS for Graph and Tree traversal:**
- Breadth-First Traversal (or Search) for a graph is similar to the Breadth-First Traversal of a tree.
- The only catch here is, that, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we divide the vertices into two categories:
- Visited nodes.
- Not visited nodes .
- A boolean visited array is used to mark the visited vertices. For simplicity, it is assumed that all vertices are reachable from the starting vertex. BFS uses a **queue data structure** for traversal.

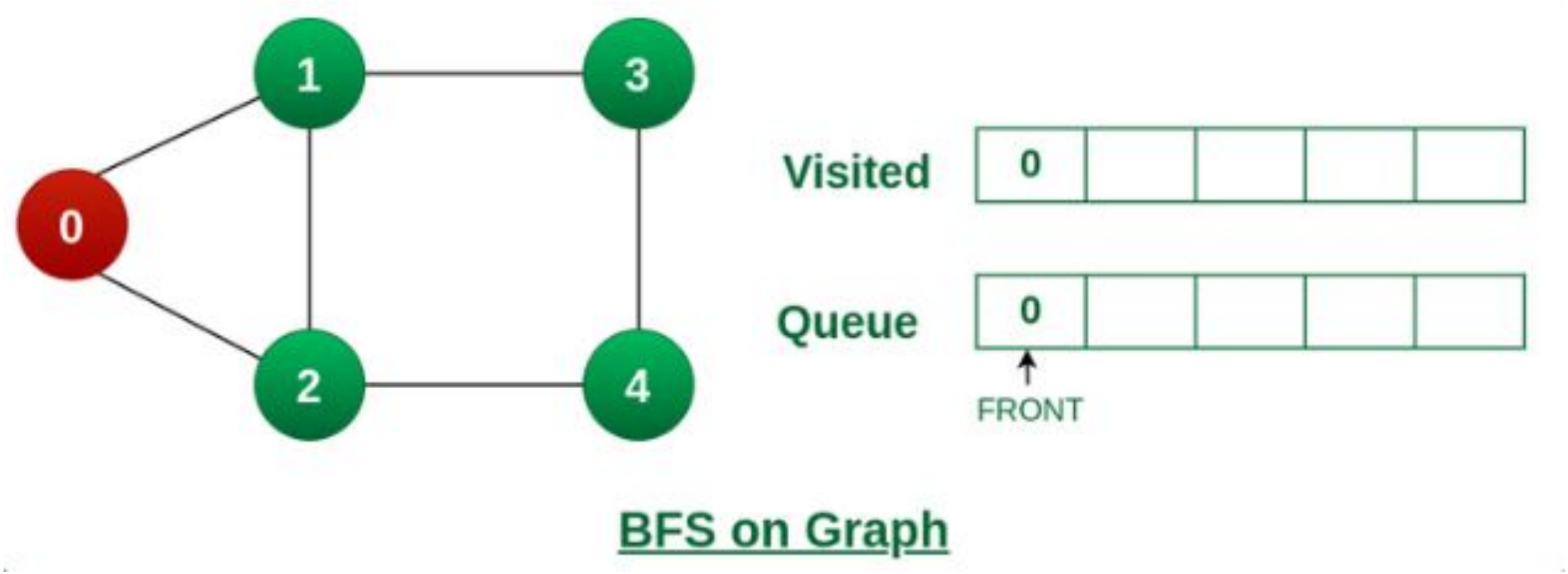
BFS Graph Traversal

Step1: *Initially queue and visited arrays are empty.*

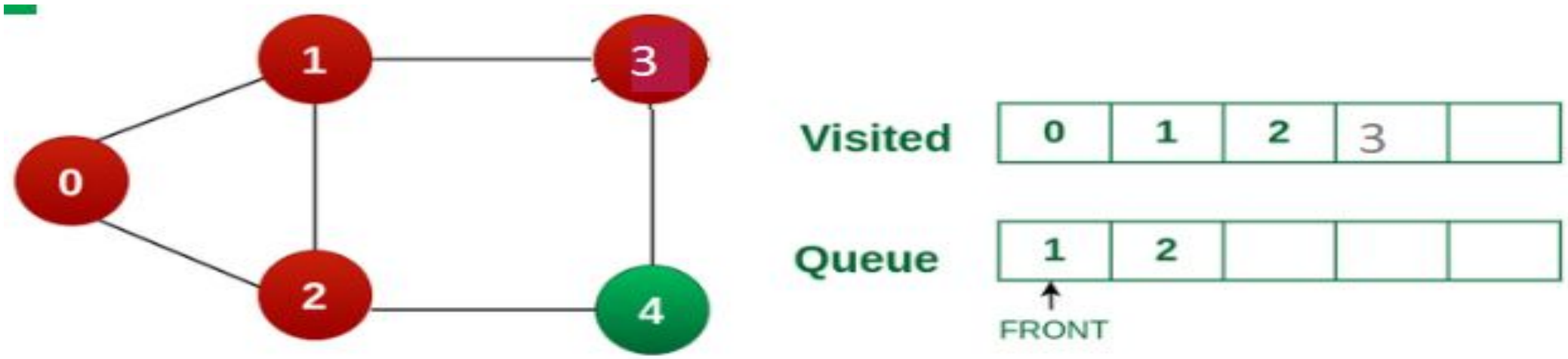


BFS on Graph

Step2: *Push node 0 into queue and mark it visited.*

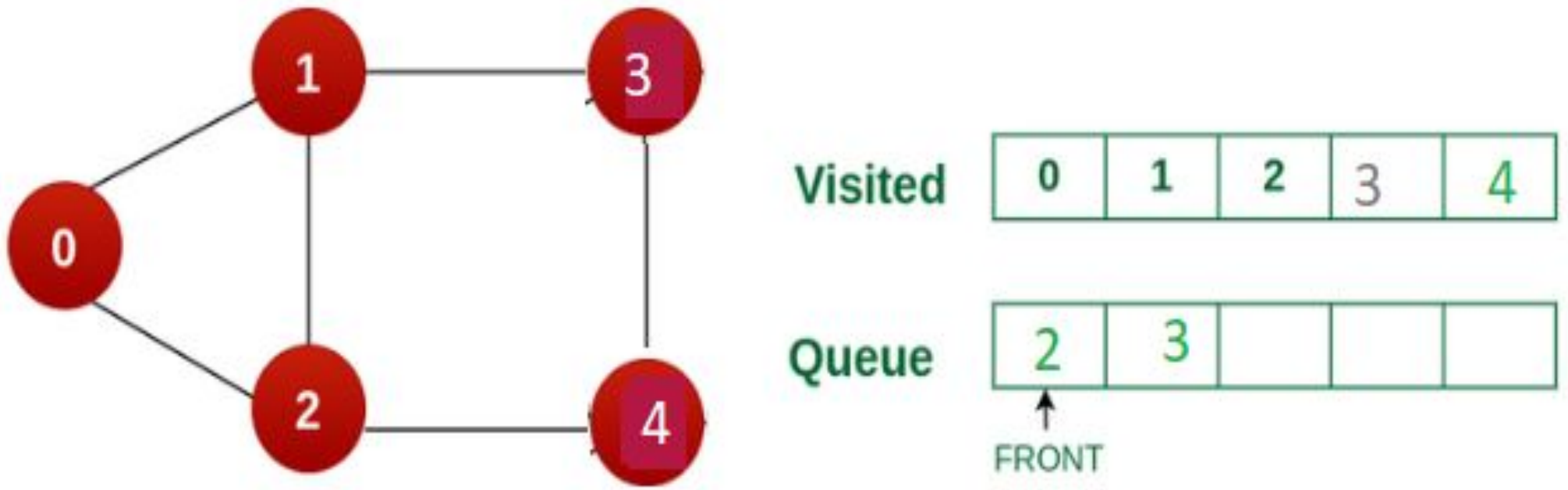


Step 3: *Remove node 0 from the front of queue and visit the unvisited neighbours and push them into queue.*



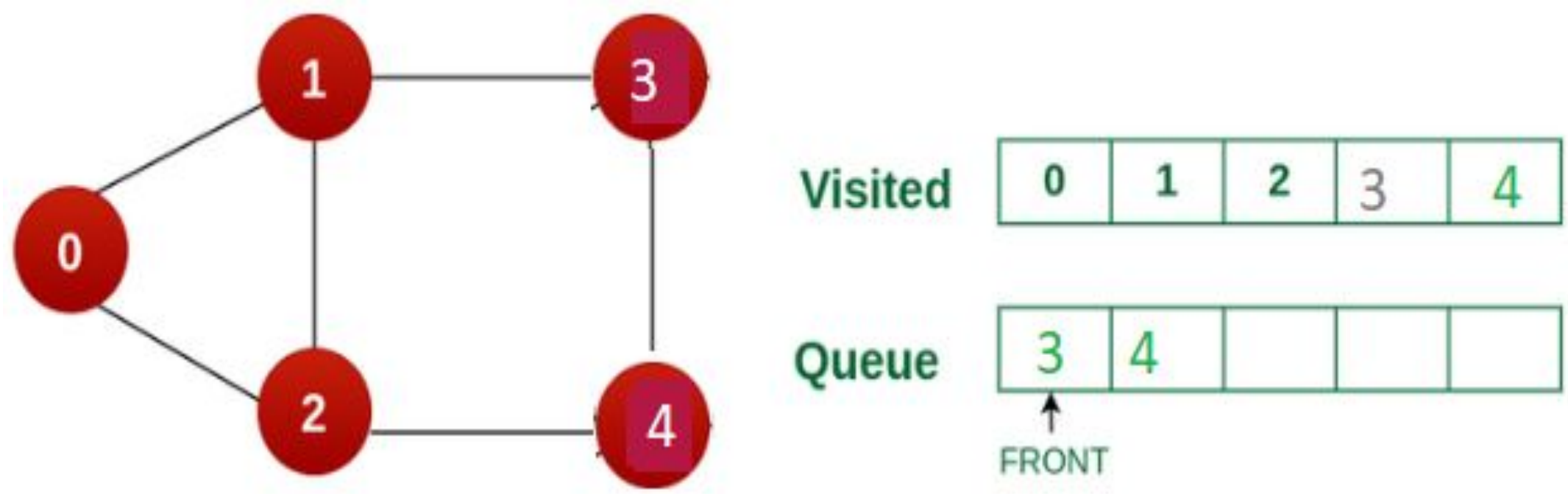
BFS on Graph

Step 4: *Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.*



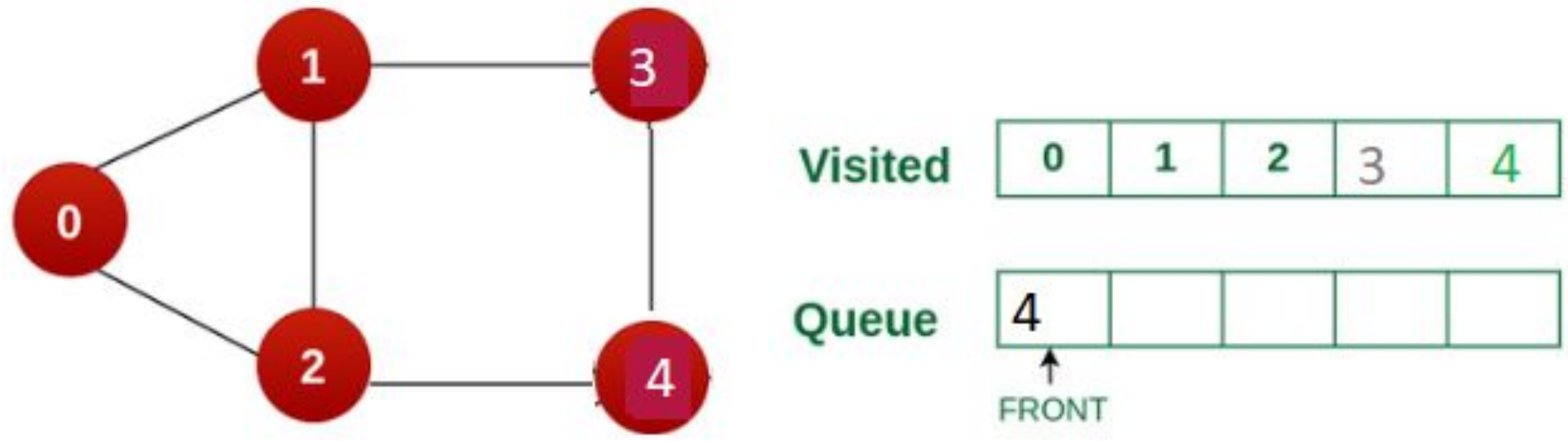
BFS on Graph

Step 5: *Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.*



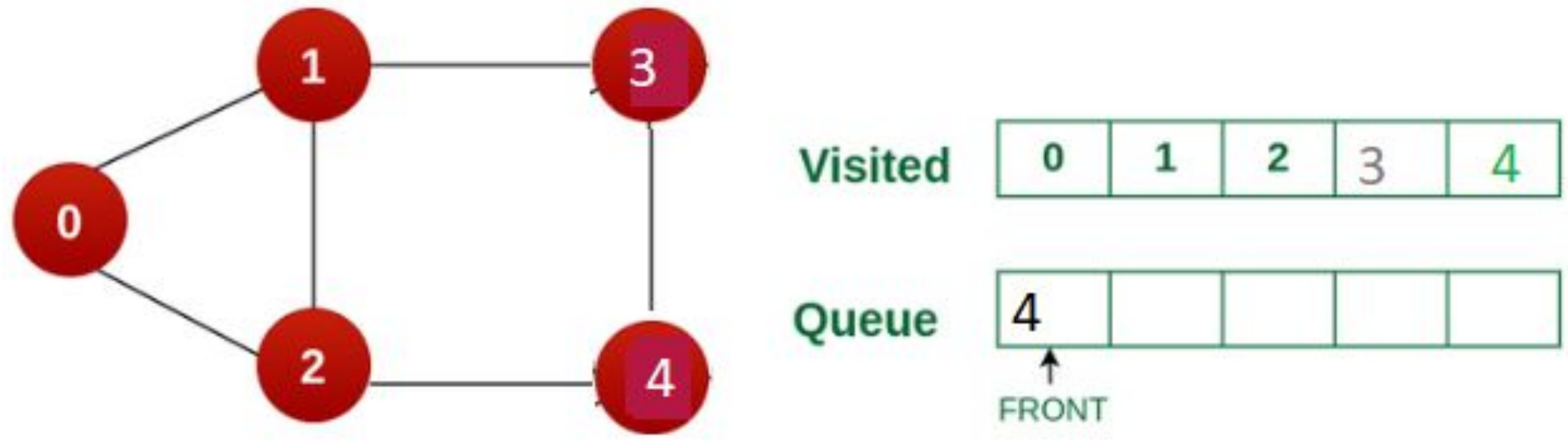
BFS on Graph

Step 6: *Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.*



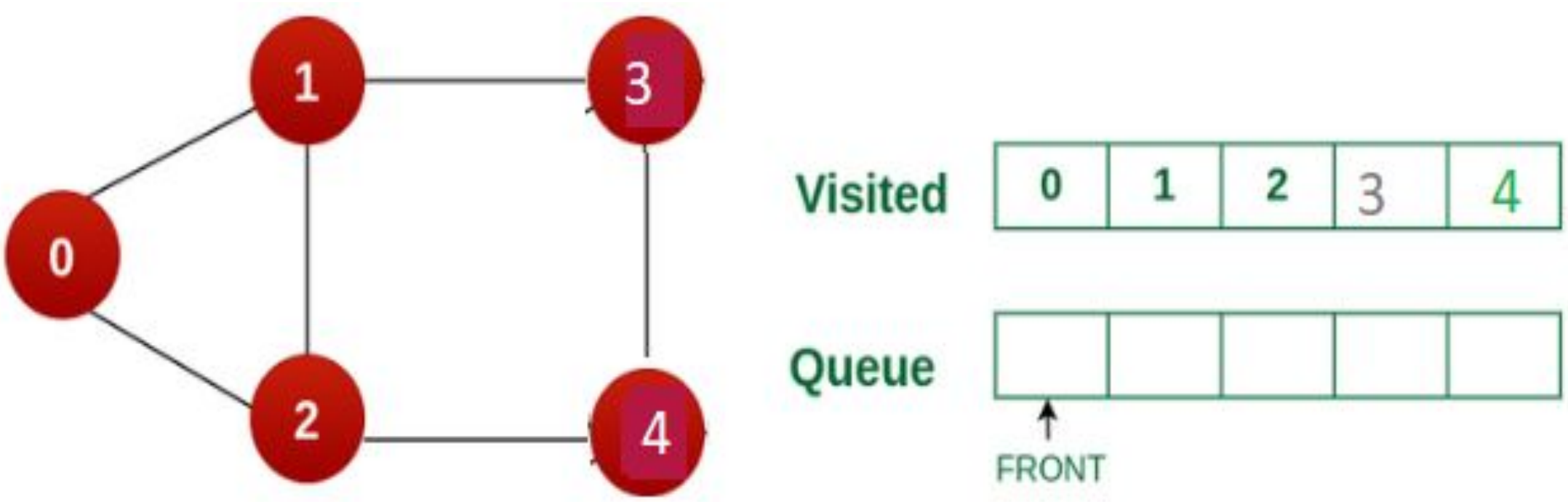
BFS on Graph

Step 7: *Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.*



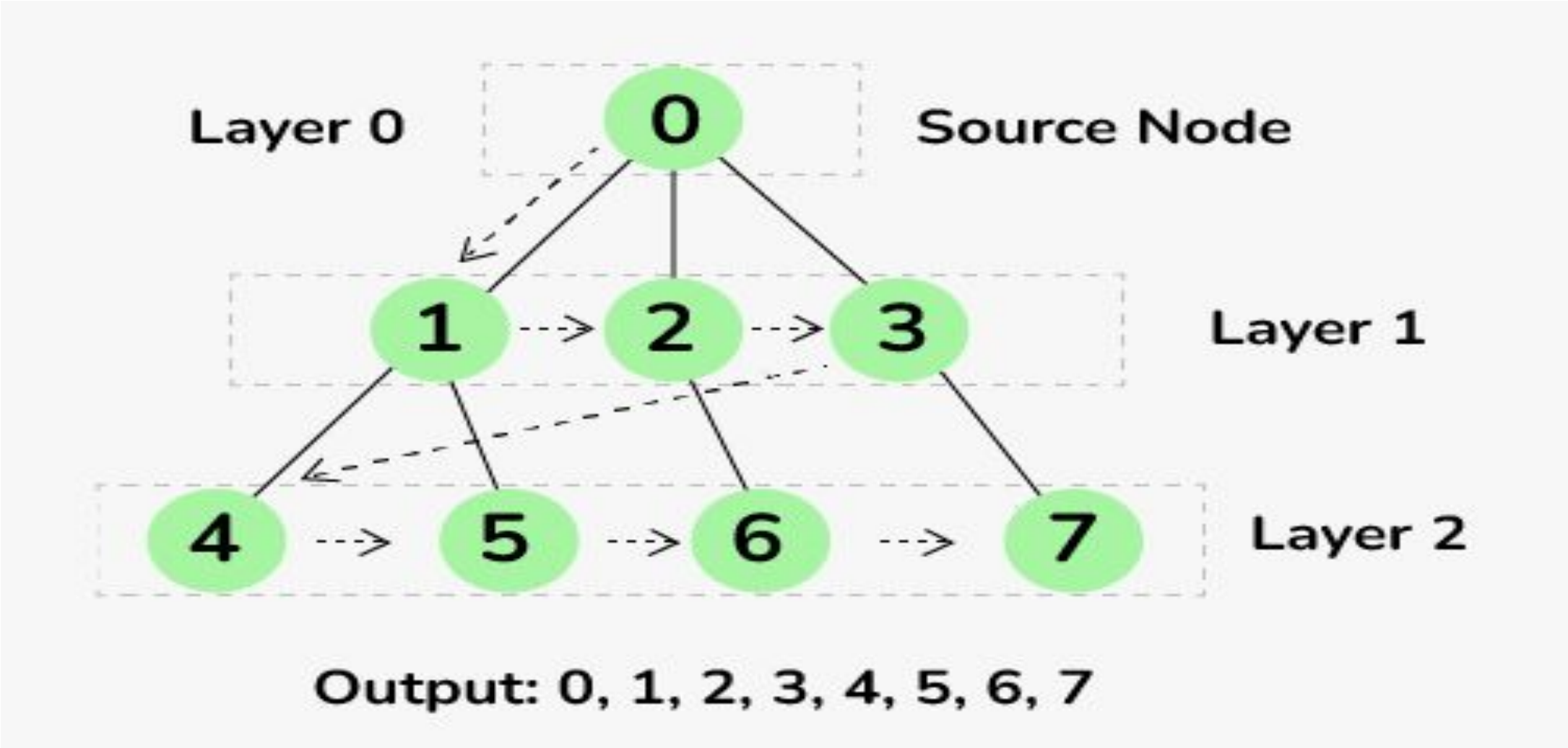
BFS on Graph

Step 8: *Now, Queue becomes empty, So, terminate these process of iteration.*

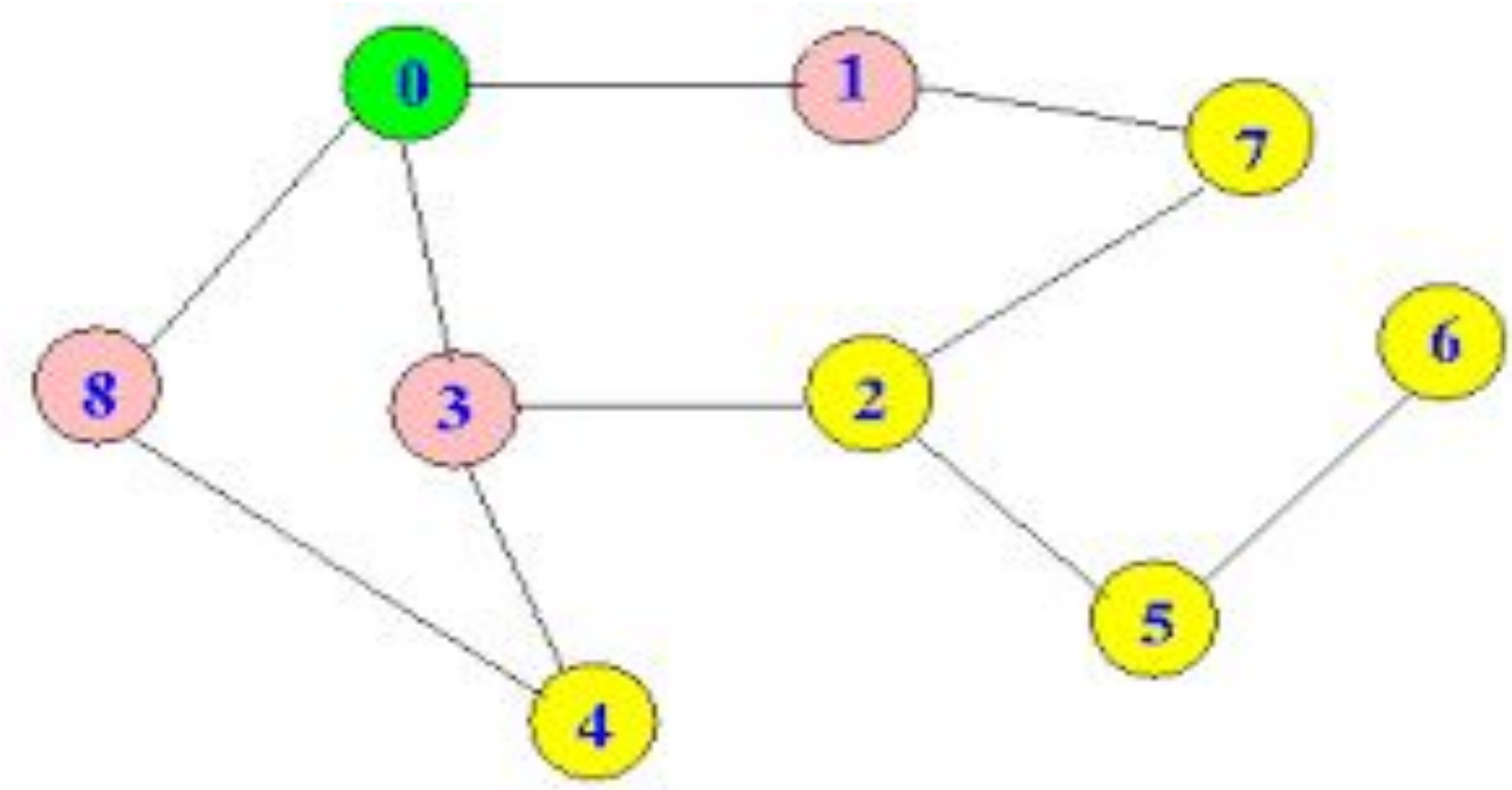


BFS on Graph

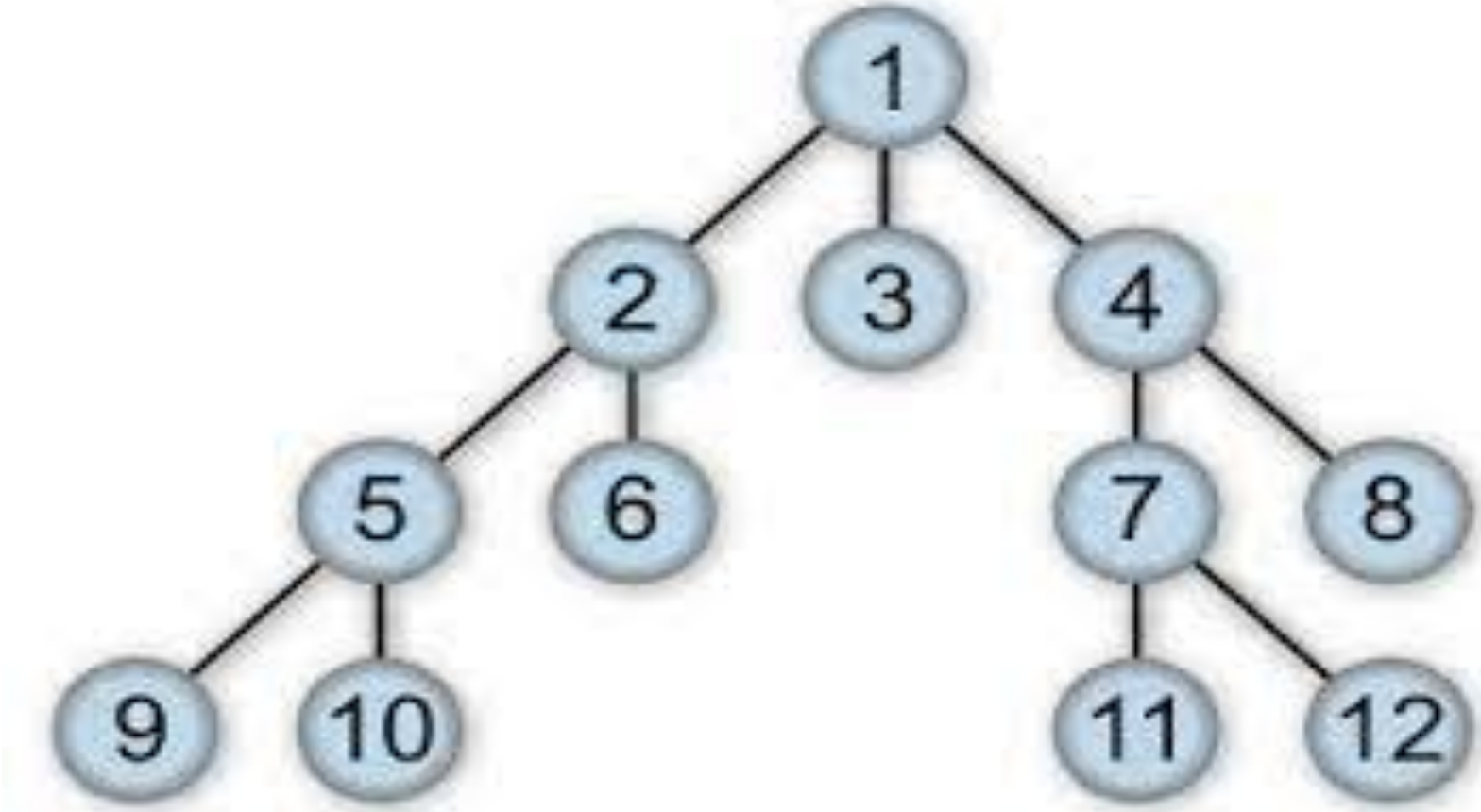
BFS- TREE Traversal



Ex No 2 Apply BFS on the following Graph



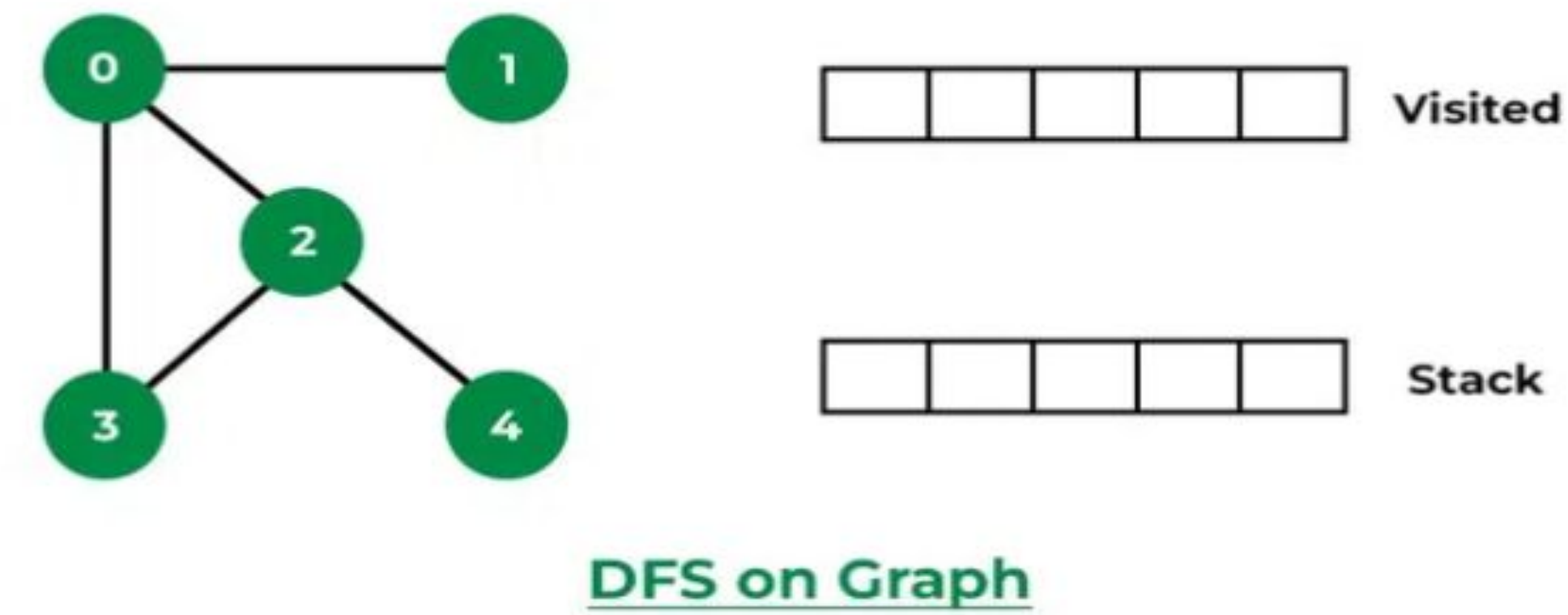
Ex no 2- Apply BFS on the following Tree



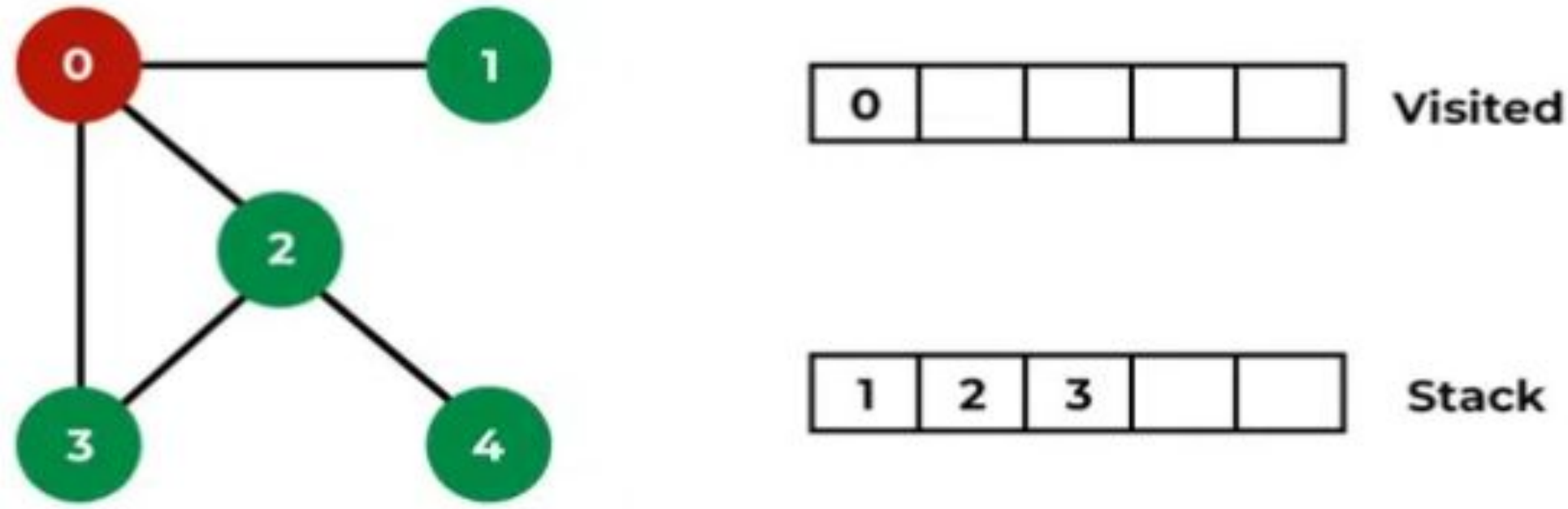
DFS Algorithm on Graph Traversal

- Depth-first search is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

Step1: *Initially stack and visited arrays are empty.*



Step 2: *Visit 0 and put its adjacent nodes which are not visited yet into the stack.*

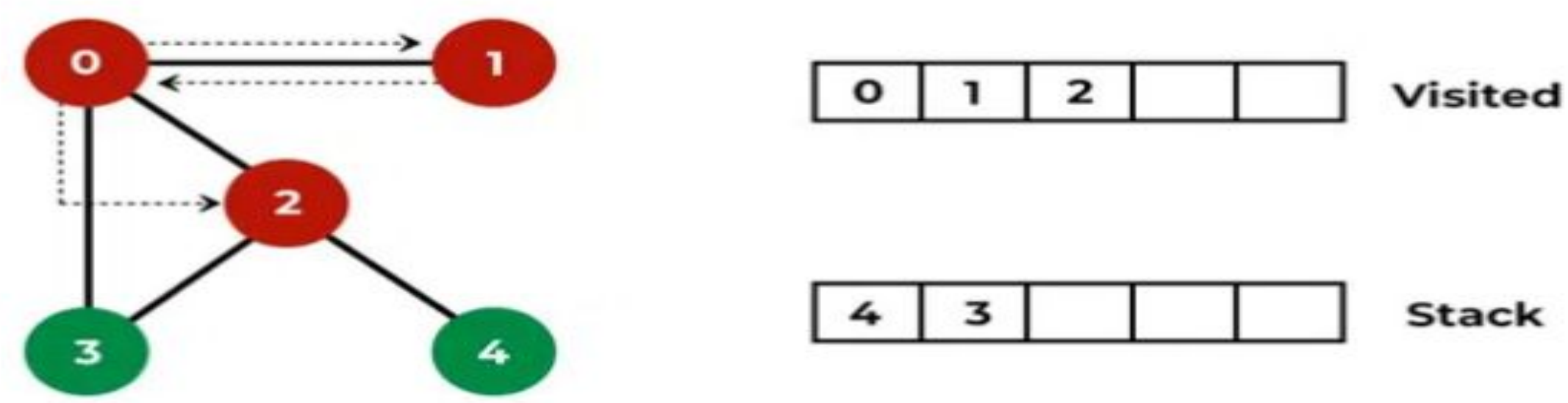


DFS on Graph

Visit node 0 and put its adjacent nodes (1, 2, 3) into the stack

Step 3: *Now, Node 1 at the top of the stack, so visit node 1 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.*

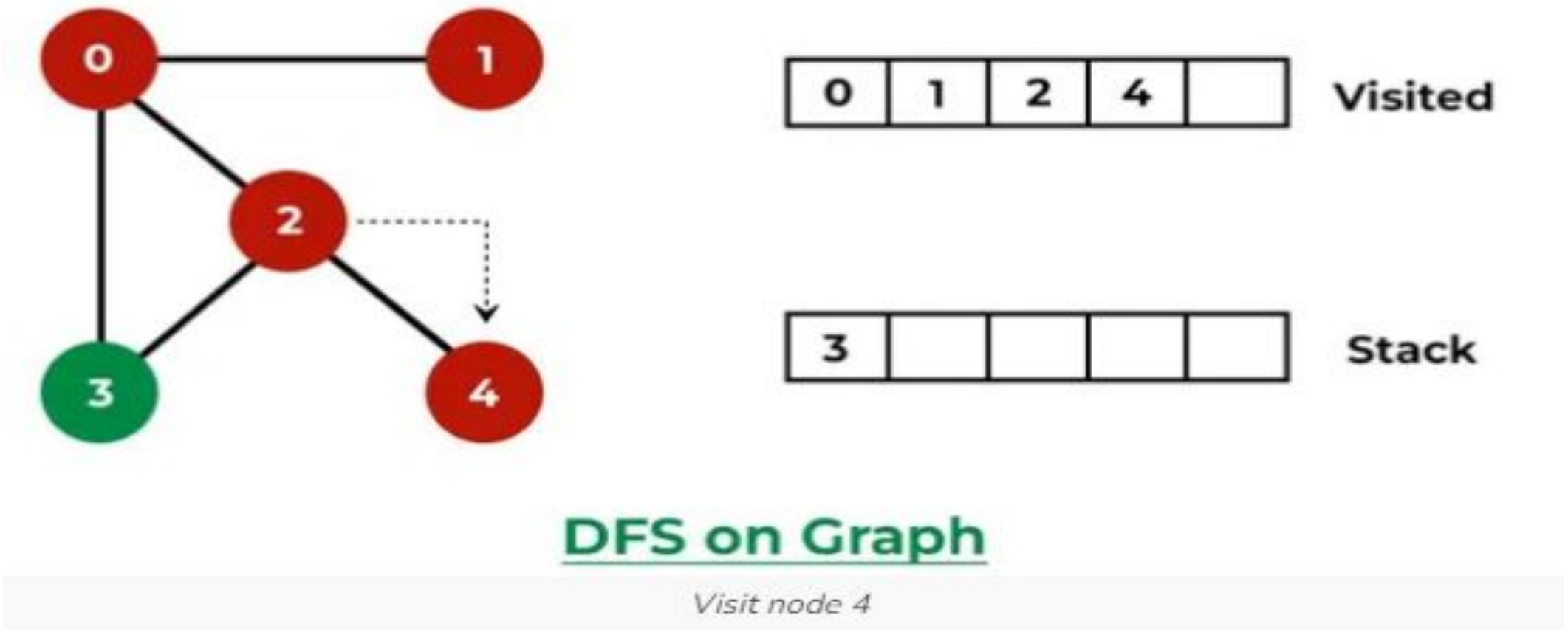
Step 4: *Now, Node 2 at the top of the stack, so visit node 2 and pop it from the stack and put all of its adjacent nodes which are not visited (i.e, 3, 4) in the stack.*



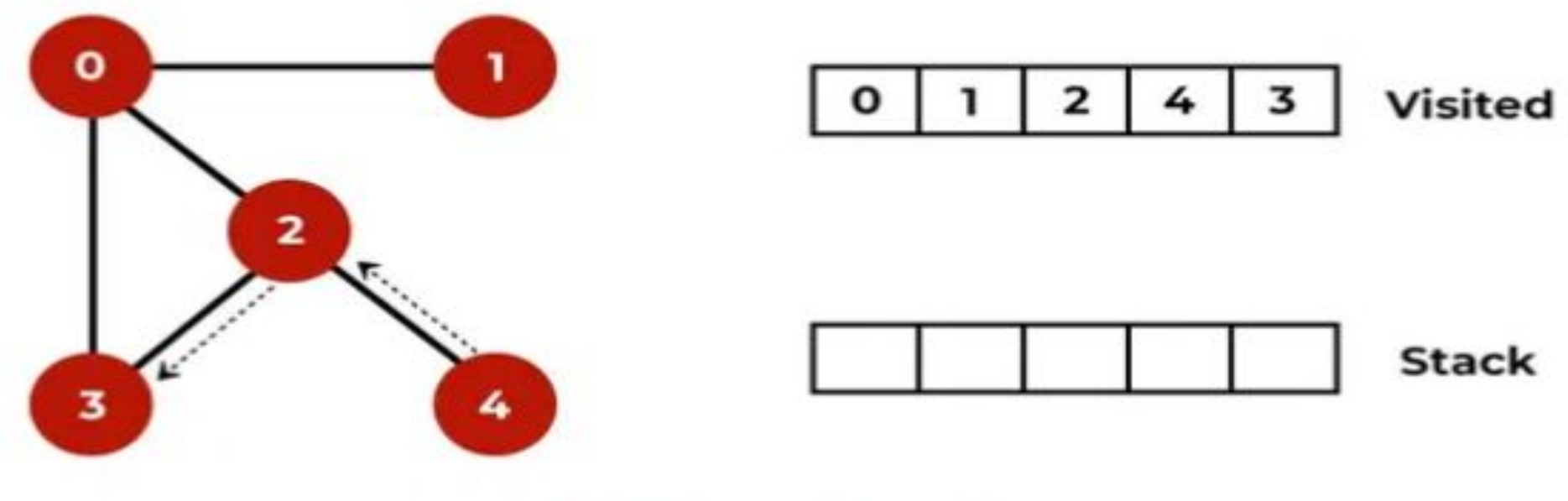
DFS on Graph

Visit node 2 and put its unvisited adjacent nodes (3, 4) into the stack

Step 5: *Now, Node 4 at the top of the stack, so visit node 4 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.*



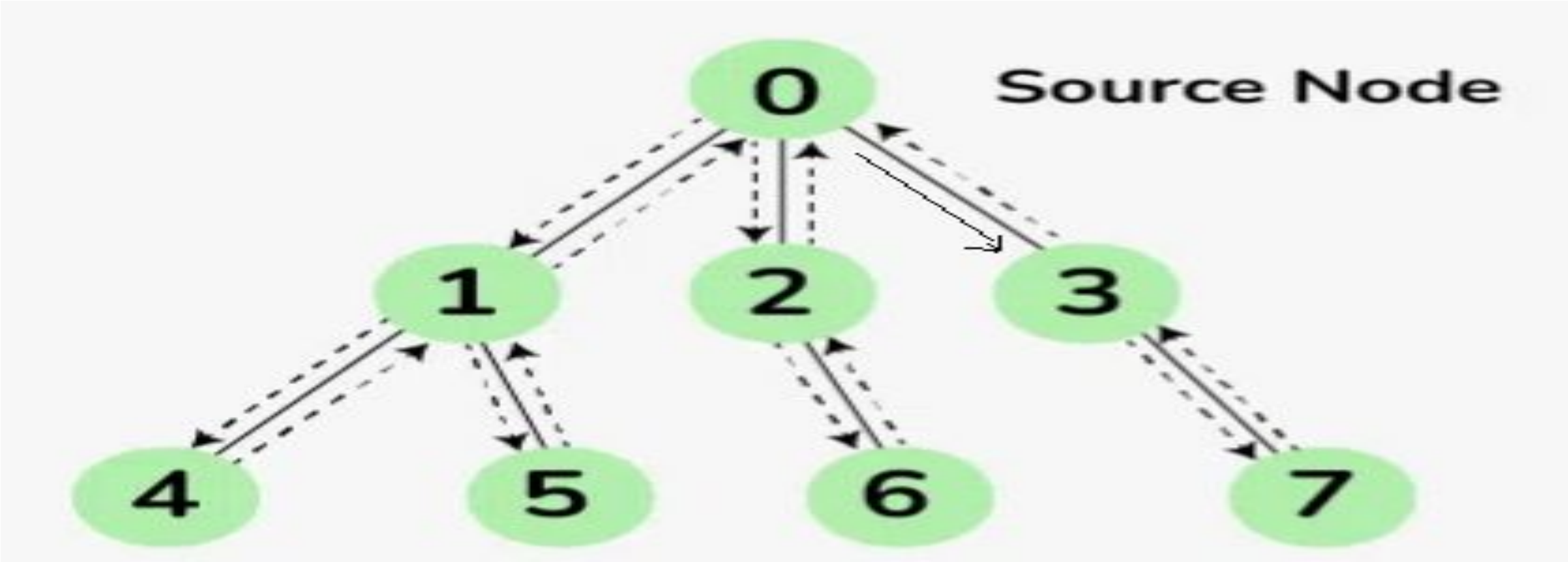
Step 6: *Now, Node 3 at the top of the stack, so visit node 3 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.*



DFS on Graph

Visit node 3

DFS on TREE Traversal



0 1 4 5 2 6 3 7